SYSTEM_ARCHITECTURE

Status: Draft v1. Consistent with PRODUCT_SPEC.

1. Principle

A modular monolith: one frontend, one backend (API process plus worker process), one Python AI service. No microservices. The most important rule: **repository code is never executed anywhere in the system.**

2. Approved technology

Layer	Choice	Reason
Frontend	React + Vite, JavaScript	Your experience, simple UI
Backend	Node.js + Express, JavaScript only	Your experience. No TypeScript unless you approve.
Database	MongoDB	Workflow state, approvals, audit
Queue	BullMQ + Redis	Your experience. Redis holds jobs only, never the real state.
AI	Python (FastAPI candidate)	Your decision. Called only by backend.
Storage	GitHub work branch (permanent) plus temporary server disk	No cloud budget. Server disk is only a temporary working copy, deleted after the job. The permanent copy of the work is the commits on the work branch in the user's repo.
Auth	GitHub OAuth	Your decision
Hosting	Render, later	Not verified: plans, prices, disk persistence.

Because there is no TypeScript, every boundary needs runtime validation (API input, job payload, AI output, GitHub response). A JSON Schema based validator (for example Ajv in Node, pydantic in Python) is the proposal. This is a new dependency and needs your approval.

3. Components

Component	Responsibility	Inputs	Outputs	Trust
frontend	Login, repo list, findings, plan, approval, diff, push permission	User actions	API calls	Untrusted client
backend API	Auth, session, ownership check, workflow state, approval check, rate limits, GitHub calls	HTTP requests	JSON, queue jobs	Trusted
backend worker	Clone, analyze, call AI, apply approved edits, build diff and manifest, static checks	Queue jobs	Files on disk, DB updates	Trusted code handling untrusted data
aiServices	Suggest findings, explanations and plan items	Code excerpts and profile	Suggestions (JSON)	Trusted service, untrusted output
MongoDB	Users, workflows, findings, plans, approvals, audit, usage			Trusted
Redis	Job queue			Trusted
Disk storage	Temporary snapshots, V2 working copy, diffs, manifests (deleted after the job)			Trusted

4. Diagram (text)

Browser -> HTTPS -> frontend -> backend API -> MongoDB and Redis backend API -> queue -> backend worker -> disk, MongoDB, aiServices, GitHub (clone only) backend API -> GitHub API (login, repo check, branch create, commits, PR)

5. Trust boundaries

1. Browser to API: everything from browser is untrusted and re-validated.
2. Repo content to our code: the repo is hostile data. It is read, never run.
3. Repo content to AI: prompt injection is assumed. AI output is a suggestion only.
4. Secrets: GitHub tokens only in backend (encrypted at rest). AI provider keys only in aiServices. Neither is ever sent to the browser or put in logs. The GitHub token now carries write access from login, so it is a higher value target.

6. Safe clone and read rules

- Clone with hooks disabled and no submodule fetching.
- Do not follow symlinks. Reject paths that leave the snapshot folder.
- Limits: total size, file count, single file size, clone time.
- Never run npm install, scripts, or any repo command.
- Parsing for static checks must use parsers that read text only.

7. Process model

- API process: handles HTTP, validates, writes state, enqueues jobs.
- Worker process: takes jobs, runs steps, updates state by conditional updates.
- aiServices: separate process, localhost in dev, private service in deployment.

8. Authorization boundaries

Every query includes the userId. A user can read only their own workflows. Banned users are blocked at middleware. Workflow actions check current state, SHA and ownership.

9. Failure modes

Failure	Handling
GitHub API down or rate limited	Retry with backoff, then clear error. State stays safe.

Failure	Handling
AI service down or slow	Show deterministic findings only. Flow does not stop.
Redis restart	Jobs re-queued from MongoDB state. No state lost.
Worker crash	Job marked failed or retried idempotently. Temp files cleaned on startup.
Clone too large	Reject with SNAPSHOT_TOO_LARGE.
SHA changed before push	Workflow becomes STALE. Push rejected.
Disk full	Reject new workflows, alert, cleanup old data.

10. Error handling rules

Every error has a code (see API_CONTRACTS). Fail closed. No silent failures. Users never see stack traces, paths or tokens. Logs carry a correlation ID.

11. Folder structure

obsidian/

- frontend/ (React + Vite): src/pages, components, api, hooks
- backend/: src/server.js (API entry), src/worker.js (worker entry), src/modules (auth, repos, workflows, analysis, findings, plans, approvals, versions, review, delivery), src/middleware, src/lib (github, storage, audit, aiClient), src/config, contracts/ (JSON Schemas), tests/, storage/ (gitignored)
- aiServices/: app/routes, prompts, schemas, tests/
- docs/

Each backend module holds its own routes, service and validation. Worker and API share models and workflow code but keep separate entry points.

12. Where AI is used and where it is not

AI	Normal code only
Project purpose and structure summary	Login, sessions, ownership, bans
Finding bugs and risks beyond simple rules	Language and framework detection
Explaining why an issue matters	State machine, approval validation
Suggesting enhancement plan items and edits	Applying edits, diff, manifest, static checks, push

13. Assumptions and unverified items

- GitHub OAuth scopes: not verified. Write access is needed from login because work is saved on a GitHub branch.
- Render capabilities, prices, disk persistence, Redis and MongoDB hosting: not verified.
- Static parser libraries per language: to be chosen.
- Express remains the choice (decided).

14. Acceptance criteria

No code path can execute repo code (code review plus test with a repo containing malicious scripts and hooks that must never fire). Secrets never appear in logs or responses. API and worker restarts lose no workflow state.